AI FOR PRODUCT MANAGERS · IIIT HYDERABAD · CAPSTONE PROJECT · PRD

SpecBridge

Jira tickets to codebase-grounded UAT test cases

Team 07: Hussain Lohawala, Qasim Naik, Rishabh Singh, Dhawal

The opportunity. Today, a product manager often sees the real user journey only after engineers have changed 3–5 repositories. If the result differs from what the PM intended, the team repeats work. SpecBridge reads the team's code and turns a Jira ticket into a grounded set of UAT test cases—including steps, expected results, affected screens, and open questions—so the PM and engineers can agree on the journey before anything is built.

1 User and job-to-be-done

Primary user

A product manager who hands UI tasks to frontend and backend engineers on a product whose frontend is split across several repositories.

Job-to-be-done

When I hand over a UI task, I want to see and agree on the exact user journey, including errors and edge cases, before engineers build it, so the first demo is what I meant.

Secondary users: frontend engineers, who can build from an agreed journey instead of guessing, and QA, who can later run the same cases.

2 Problem

How it works today

1. The PM gives a UI task to the frontend and backend engineers.
2. The frontend engineer translates it into changes across 3–5 repositories.
3. Only when the work is done does the engineer demo the user journey to the PM.
4. If the result is not what the PM meant, the cycle repeats, sometimes several times.

Why this happens

The journey is never made concrete and agreed on before development. The PM describes the goal in product language (for example, "let users finalize a bill") but may not know which screens and flows across repositories the change touches. Mismatches, missing error states, and edge cases therefore surface during the demo—one of the most expensive moments to discover them.

Why now

Current LLMs can work across plain-language requirements and code, while the estimated cost of processing a ticket is only a few cents. Research on repository-level code retrieval also shows both the value of retrieving relevant context and the difficulty of consistently finding the right evidence [1, 2].

3 Approach

SpecBridge converts a ticket into a reviewed, code-grounded UAT page through a controlled agent workflow.

Step	Workflow stage
1	Ticket URL is submitted.
2	The system understands the ticket: stated and inferred acceptance criteria, affected screens, open questions, and risks.
3	An orchestrator coordinates the work.
4	Specialized agents work on each acceptance criterion.
5	Results are merged and independently reviewed.
6	A draft UAT page is published to Confluence for human review [7, 8].

1. **Understand the ticket.** Extract stated and inferred criteria, screens, questions, and risks. If a ticket is unclear or unsafe, return questions rather than inventing test cases.
2. **Retrieve code evidence.** Agents search a code knowledge base using keywords, local embeddings, and links between files across repositories. This hybrid retrieval approach is motivated by code-retrieval research, which finds that retrieving useful repository context remains a challenge [1, 2]. Tools are read-only, and generated cases cite the code used as evidence.
3. **Review independently.** A reviewer checks acceptance-criteria coverage and verifies that each cited file and screen exists. Weak or incomplete criteria can be sent for another review round. LangGraph supports workflows that pause for human input and resume after a decision [5].

Proposed implementation stack
Model: Gemini 3.8 Flash via OpenRouter, configurable so the model can be replaced.
Orchestration: LangGraph.

4 Example: from ticket to test cases

TICKET AS WRITTEN BY THE PM

“Start visit” does nothing from patient search

When a front-desk user searches for a patient and clicks **Start visit** on a result, nothing happens. Expected: the visit form opens for that patient and the visit can be started.

What SpecBridge returns (illustrative)

Affected screens: patient search results, where the button appears, and the start-visit form. The button is built in the patient-chart repository and shown in the patient-management search screen, so two repositories are involved.

TC-01 · **Happy path**
Steps: Log in as front desk. Search for a patient with no active visit. Click **Start visit** on the result.
Expected result: The start-visit form opens for that patient. After saving, the visit is shown as active.

TC-02 · **Error path**
Steps: Repeat the same steps while the server rejects the request.
Expected result: An “Error starting visit” message appears, and no visit is created.

Question for the PM: If the patient already has an active visit, should the button be hidden or should it open that visit?

Why it matters: The PM reviews this before any code is written. A mismatch is caught before the demo instead of triggering another development cycle.

5 Build vs. rules

Why rules alone are not enough

- **Free-text input.** Tickets vary from formal acceptance criteria to informal Hinglish. Templates can enforce structure, but not correctness.
- **Language-to-code mapping is semantic.** "Finalize a bill" must be mapped to the right code among 2,322 files across 5 repositories. On golden tickets that do not name files, keyword search plus code links finds a changed file in the top 10 for 74% of tickets. Adding AI search by meaning raises this to 79%. The agent can then continue searching on its own.
- **Reasoning, not just lookup.** Inferring unstated criteria, negative paths, and contradictions requires judgment.

Where deterministic rules remain essential

Team-only ticket intake; secret redaction; read-only AI tools; verification of every cited file and screen; publishing performed only by code; and explicit cost and time caps. These controls address documented risks such as prompt injection, sensitive-information disclosure, and excessive agency [3, 4].

Why AI is worth it

An AI draft is estimated to cost about $0.02–0.07 and take 1–2 minutes per ticket, based on measured token sizes. This compares with an assumed 30–60 minutes of PM time per ticket today, while also providing code knowledge the PM may not have. The PM time-saving estimate is an assumption to be measured in customer sessions.

Accuracy needed and cost of errors

Error	Potential cost	Tolerance
Leaks a secret or obeys instructions hidden in a ticket	Security incident	ZERO
Invents a screen or flow that does not exist	Wasted PM/developer time; loss of trust	LOW
Misses a case, such as an error state	Later bug and rework; PM review catches many gaps	SOME
Awkward wording or a duplicate case	Minor editing	SOME

The output is a draft that a human approves, so it need not be perfect. Code grounding and safety, however, must be near-perfect.

6 Risks and mitigations

Risk	Mitigation
Hallucinated screens, steps, or code references	Every case must cite the code it used. The reviewer verifies each cited file, line, and screen name against the code.
Prompt injection or secrets inside tickets	Treat ticket text as data; remove secrets before the AI sees them; give AI read-only tools; allow only code to publish.
Vague or contradictory tickets	A gate converts ambiguity into questions for the PM instead of guessed test cases.
Search misses the right code (79% hit rate on fair tickets today)	Agents search again per criterion; misses are logged and addressed in improvement rounds.
The build drifts from the agreed journey	The approved page becomes the engineer's direction and the demo checklist, making drift easier to catch step by step.
Overwriting the PM's edits on re-runs	Updates never overwrite human edits; new drafts are added as page comments.

Risk	Mitigation
Cost or latency runaway; free-tier model capacity	Cap processing at $0.25 and two review rounds per ticket; fall back from free to paid capacity.
Evaluation results are overly flattering	Freeze the code version; use real tickets fixed after that date and an answer key based on actual fixes; remove links to fixes; score 18 file-naming tickets separately and include 6 adversarial tickets.

7 Success metrics

Outcome

Reduce the number of demo-and-rework rounds per UI task. Establish today's baseline through customer interviews.

Primary capstone metric

The share of generated test cases that PMs keep without edits, indicating that the journey was correct before development.

Evaluation plan

Measure this in at least 5 sessions with real PMs, engineers, or QA participants (not classmates), and capture verbatim feedback.

Additional measures

Track mismatches surfaced before development (per PM), engineer clarity rating on a 1–5 scale, tokens used, cost per ticket, and processing time. LLM-as-a-judge agreement should remain a secondary signal: empirical software-engineering research finds its alignment with human ratings varies by task and evaluation method [6].

Test bed: OpenMRS 3, using 5 connected React repositories frozen at 31 March 2026. The golden set includes 63 real tickets fixed after that date, with an answer key derived from actual code changes, plus 6 hand-written adversarial tickets. OpenMRS publicly maintains the O3 reference distribution and frontend repositories, making it a traceable open-source test bed [9]. The team will select and label the final 30–50 tickets.

8 Proposed ship gate

Targets below are proposed and must be confirmed by the team.

Measure on golden set	Target
Acceptance criteria covered by at least one test case	$\geq 90\%$
Cited files and screens verified in code (grounding)	$\geq 95\%$
Adversarial tickets handled safely	6 / 6
Human rubric from 2 raters (accuracy, grounding, usefulness)	$\geq 4/5$
LLM judge agreement with humans on spot-checks	$\geq 80\%$
Cost / time per ticket	$\leq \$0.10/3min$

9 References

[1] Z. Z. Wang, A. Asai, X. V. Yu, F. F. Xu, Y. Xie, G. Neubig, and D. Fried, “CodeRAG-Bench: Can Retrieval Augment Code Generation?” *Findings of NAACL 2025*, pp. 3199–3214. Supports retrieval for code tasks and documents the challenge of selecting useful repository context. [Online]

[2] J. Strich, F. Schneider, I. Nikishina, and C. Biemann, “On Improving Repository-Level Code QA for Large Language Models,” *ACL 2024 Student Research Workshop*. Studies repository-level code understanding using retrieval-augmented pipelines and file dependencies. [Online]

[3] OWASP GenAI Security Project, “OWASP Top 10 for Large Language Model Applications, 2026.” Identifies prompt injection, sensitive information disclosure, excessive agency, and related LLM application risks. [Online]

[4] C. Autio et al., *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile*, NIST AI 600-1, National Institute of Standards and Technology, 2024. A risk-management reference for trustworthy design, development, evaluation, and use of generative AI. [Online]

[5] LangChain, “LangGraph: Human-in-the-loop / Interrupts.” Official documentation describing workflows that pause for human review and resume after a decision. [Online]

[6] “Can LLMs Replace Human Evaluators? An Empirical Study of LLM-as-a-Judge in Software Engineering,” *Proceedings of the ACM on Software Engineering*, 2025. Finds judge-to-human alignment varies by task and evaluation method. [Online]

[7] Atlassian, “Jira Cloud Platform REST API: Issue fields.” Official API documentation for Jira issue fields and related configuration. [Online]

[8] Atlassian, “Confluence Cloud REST API: Create page.” Official API documentation for creating Confluence pages, subject to permissions and API scopes. [Online]

[9] OpenMRS, official GitHub organization and repositories, including the OpenMRS 3 reference distribution and frontend modules. [Online]

Out of scope: writing or changing code, automated tests, backend design, running the app, and auto-approving pages. Internal project claims and measurements: team proposal (October 2026), project build log, and measured search scores (Step 4, 27 October 2026). External research and documentation are listed in Section 9.